Week 3 Assignment Report

CS7CS4 | Machine Learning

Hera Khan | Student ID: 22312194 | Trinity College Dublin

Dataset Identification

```
# id:8-8-8
```

There are three columns of data that have the following preview below:

```
      0     1         2
0  0.28  0.37  0.736692
1 -0.13  0.41  0.530386
2 -0.25 -0.36 -0.141935
3  0.57 -0.46  0.115615
4 -0.26 -0.84 -0.700020
```

Here the columns 0,1 are served as the X inputs- X1,X2 respectively. The column 2 is used as the mapped output- Y.

(i) Lasso and Ridge Regression

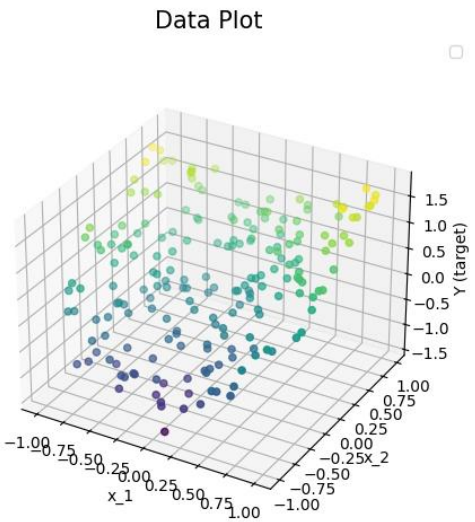
(i)(a) 3D scatter plot of the data

A 3D scatter: x1 on the x-axis, x2 on the y-axis, Y (target values) on the z-axis. The box_aspect was shrunk in order to show the Z-axis label.

The data lies on a curve. This is noticeable from the point differently coloured gradients where the colours twist and curve across the surface, which typically indicates a quadratic relationship (like a parabola/paraboloid) or an interaction effect between x1 and x2. In order to make sure this effect was achieved, I had scaled the colour gradient to follow the value of y hence:

```
ax.scatter(X1, X2, y, c=y, cmap='viridis')
```

[Figure 1: 3D scatter of the training data]



Caption: Main scatter plot showing x1, x2 projected with their target Y on a 3d plane

(i)(b) Polynomial features and Lasso for a range of C

The features were built through the in-built functions provided by python- PolynomialFeatures.

```
poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)

feature_names = poly.get_feature_names_out(['x_1', 'x_2'])
print("Feature names:", feature_names)
```

Features were crosschecked accordingly and reported via the print statement. This gives us new inputs in X via squaring or multiplying with each other. To capture a curved surface, this is necessary.

This gave me 20 features in total and in order to display those functins, I used get_feature_names_out to crosscheck all possible combinations. My C to alpha conversion was:

```
alpha = 1/(2*C)
```

All possible C values in a range enclosed by a loop from [0.001, 0.01, 0.1, 1, 10, 100, 1000] after noticing how menial the 1-on-1 conversion was. After this I cleaned up my data using *pandas,* which allowed me to present my data more organically and in a far more organized manner.

```
Trained C = 0.001 (alpha = 500.0000) -> Intercept: 0.3855
Trained C = 0.01 (alpha = 50.0000) -> Intercept: 0.3855
Trained C = 0.1 (alpha = 5.0000) -> Intercept: 0.3855
Trained C = 1 (alpha = 0.5000) -> Intercept: 0.3855
Trained C = 10 (alpha = 0.0500) -> Intercept: 0.2075
Trained C = 100 (alpha = 0.0050) -> Intercept: 0.0602
Trained C = 1000 (alpha = 0.0005) -> Intercept: 0.0493
```

The printed output has been screenshotted below:

Feature							
x_1	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0831
x_2	0.0	0.0	0.0	0.0	0.8285	0.9523	1.0164
x_1^2	0.0	0.0	0.0	0.0	0.4807	0.9096	0.9731
x_1 x_2	0.0	0.0	0.0	0.0	0.0000	0.0000	0.1068
x_2^2	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0000
x_1^3	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0000
x_1^2 x_2	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0000
x_1 x_2^2	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0321
x_2^3	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0000
x_1^4	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0000
x_1^3 x_2	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0000
x_1^2 x_2^2	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0294
x_1 x_2^3	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.2046
x_2^4	0.0	0.0	0.0	0.0	0.0000	-0.0093	-0.0502
x_1^5	0.0	0.0	0.0	0.0	0.0000	-0.0267	-0.2327
x_1^4 x_2	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0003
x_1^3 x_2^2	0.0	0.0	0.0	0.0	0.0000	0.0000	0.2465
x_1^2 x_2^3	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.1307
x_1 x_2^4	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0000
x_2^5	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0745

[Image 1: Table of coefficients that shows the features]

From the above picture it can be discerned that the readable non-zero coefficients are:

--- FINAL COMPARISON TABLE ---

Feature	C = 0.001 (R²: 0.000)	C = 0.01 (R²: 0.000)	C = 0.1 (R²: 0.000)	C = 1 (R²: 0.000)	C = 10 (R²: 0.852)	C = 100 (R²: 0.920)	C = 1000 (R²: 0.928)
x_1	0.0	0.0	0.0	0.0	0.0000	0.0000	0.0831
x_2	0.0	0.0	0.0	0.0	0.8285	0.9523	1.0164
x_1^2	0.0	0.0	0.0	0.0	0.4807	0.9096	0.9731
x_1 x_2	0.0	0.0	0.0	0.0	0.0000	0.0000	0.1068
x_1 x_2^2	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0321
x_1^2 x_2^2	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0294
x_1 x_2^3	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.2046
x_2^4	0.0	0.0	0.0	0.0	0.0000	-0.0093	-0.0502
x_1^5	0.0	0.0	0.0	0.0	0.0000	-0.0267	-0.2327
x_1^4 x_2	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0003
x_1^3 x_2^2	0.0	0.0	0.0	0.0	0.0000	0.0000	0.2465
x_1^2 x_2^3	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.1307
x_2^5	0.0	0.0	0.0	0.0	0.0000	0.0000	-0.0745

[Image 1: Table of non-zero coefficients that shows the features]

Feature	**C = 10**	**C = 100**	**C = 1000**
R score	0.852	0.920	0.928
Non-zero parameters	x2, $x1^2$	x2, $x1^2$, $x2^4$, $x1^5$	x1, x2, $x1^2$, x1·x2, $x1.x2^3$, $x2^4$, $x1^5$, $x1^3.x2^2$, $x1^2.x2^3$, $x2^5$

The parameters change as C increases because:

- Small C means a heavy L1 penalty - α, so most or all parameters are exactly zero, and the model predicts roughly a constant. This is good to prevent overfitting but bad when the model is underfitting as is.
- At large C the penalty is weak, so many parameters are non-zero and their values get larger. See table above where at C=1000, all 10 parameters out of 20 come out non-zero which would constitute as overfitting.

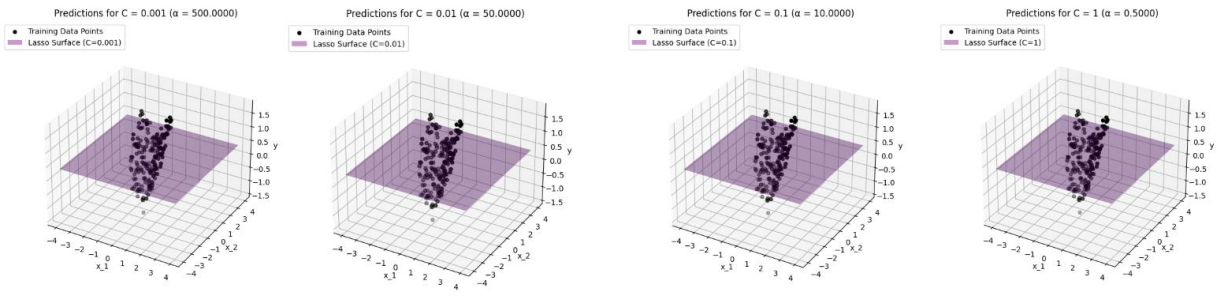
The features x2, $x1^2$ change first and foremost so those features are the ones the model considers the most important.

All 5 features have very different scales, so the penalty treats them unevenly. R_score was imported and used because it provides a clear idea on how good the model is from 0 to 0.1. It's unbiased and isn't manipulated by the type of model its being scored against.

(i)(c) Predictions on a grid

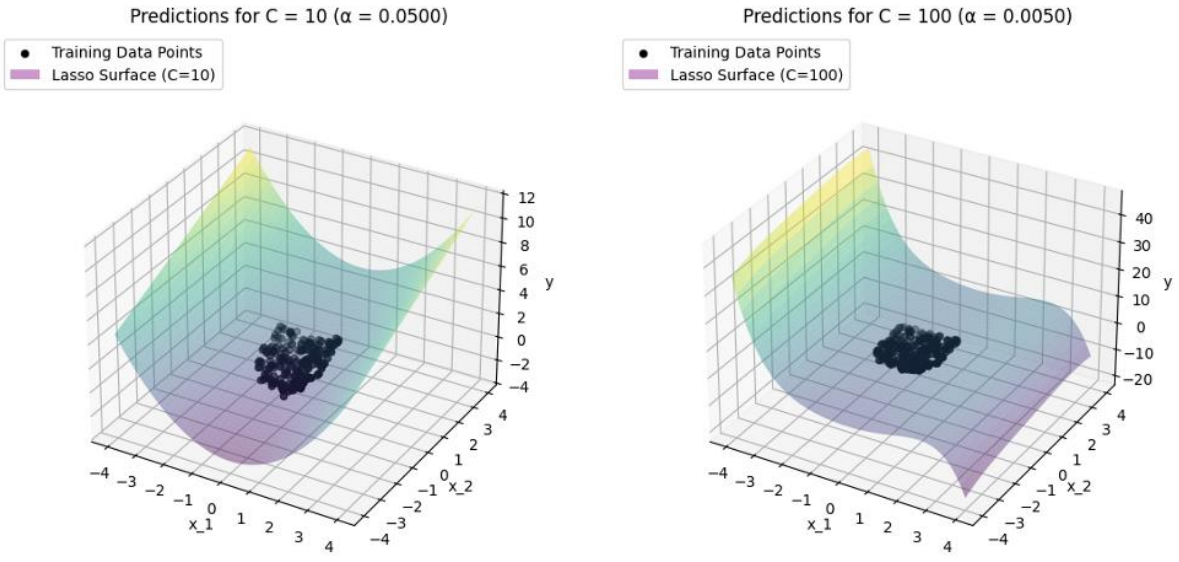
Nested-loops were used so that the grid goes through the same `PolynomialFeatures` transform before `predict`.

[Figure 2: Lasso predictions (surface) and training data (points) for C = [0.001], [0.01], [0.1], [1]]



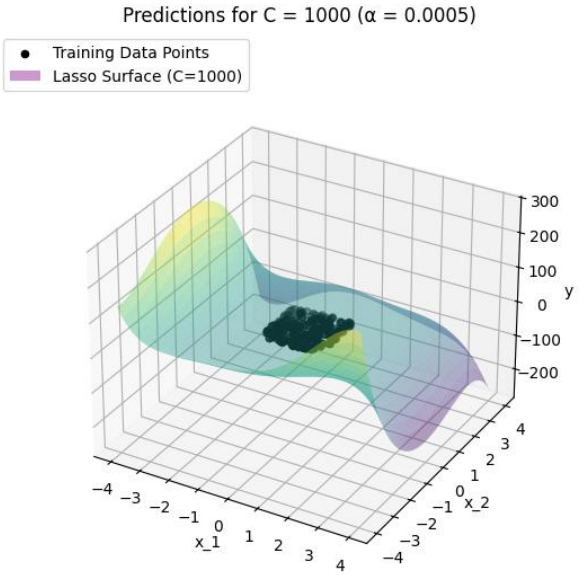
Caption: These plots were printed separately but were displayed together as there is no change in the model prediction. All of them perform underfitting against the actual real data.

[Figure 3: Lasso predictions (surface) and training data (points) for C = [10],[100]]



Caption: These plots were printed separately but were displayed together as these seem to be the sweet spot where the data model performs decently well. Here the features that actually matter - $x2, x1^2$ *are shown in full display that they affect the curvature.*

[Figure 4: Lasso predictions (surface) and training data (points) for C = [1000]]



Caption: This plot specifically shows the consequences of overfitting a model. Here due to the many non-zero parameters the model fails to capture the true essential curvature of the actual data model that is important.

(i)(d) Under- and over-fitting

As discussed under the captions of figures 2 and 4, patterns of overfitting and underfitting are noticed in the model. Defining underfitting and overfitting as:

- **Underfitting:** Where the model is too simple to capture the pattern, so it has large errors even on the training data. This is especially visible in values of C = [0.001,0.01,0.1,1] where the flat plane model cannot work with the curvature, thus ignoring it because Lasso has zeroed out all the coefficients.
- **Overfitting:** The model is so flexible it follows the noise in the training data, so it fits the training points well but predicts badly on new points. In overfitting, the regularization penalty becomes nearly non-existent which leads to the model capturing the core quadratic points of the data. This is great to visualize how the actual training data is behaving but it is awful with predicting new targets. Where there is no training data the readings will come off erratically.

As shown in the highlighted row down below in the table , small C gives few or zero parameters and a flat surface (underfit); large C gives many parameters and a surface that bends to individual points (overfit); a middle C gives the right amount of curvature. This is especially visible in figure 3 where the C = 10, C = 100 provides the perfect middle ground for model to capture the curvature while also being regularized enough to act for the predictions.

(i)(e) Ridge regression

I copied the near entire Lasso code and replaced the model from Lasso to Ridge. Everything else remains the same but due to L2 regularization, there will be more non-zero coefficients.

```
Trained C = 0.001 (alpha = 500.0000) -> Intercept: 0.3659
Trained C = 0.01 (alpha = 50.0000) -> Intercept: 0.2732
Trained C = 0.1 (alpha = 5.0000) -> Intercept: 0.1443
Trained C = 1 (alpha = 0.5000) -> Intercept: 0.0658
Trained C = 10 (alpha = 0.0500) -> Intercept: 0.0287
Trained C = 100 (alpha = 0.0050) -> Intercept: 0.0267
Trained C = 1000 (alpha = 0.0005) -> Intercept: 0.0270
```

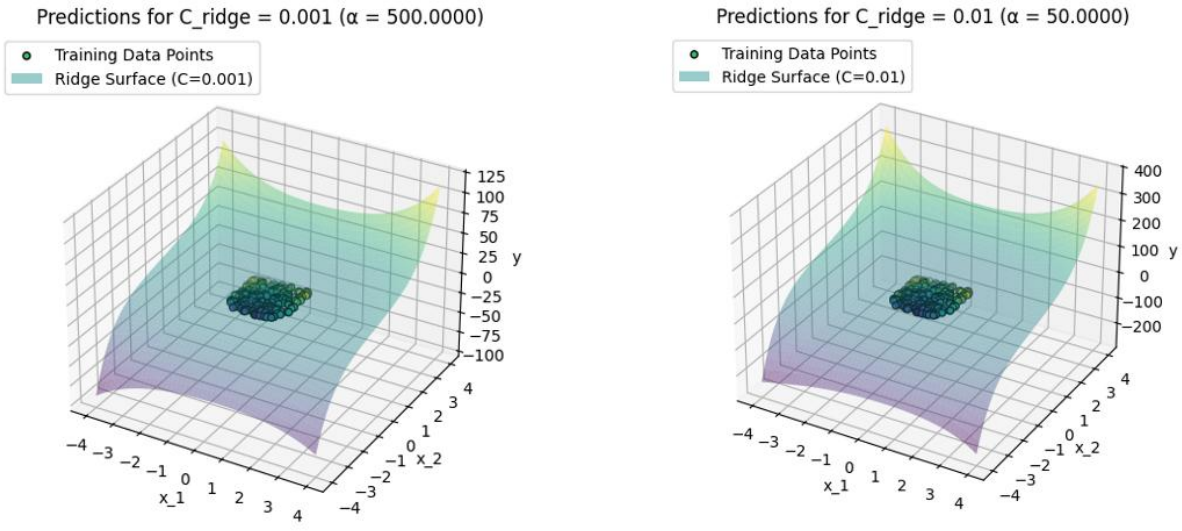
The code to compare non-zero values against zero values changes dramatically with the final filtered features being all 20 features.

--- FINAL COMPARISON TABLE ---

Feature	C = 0.001 (R^2: 0.275)	C = 0.01 (R^2: 0.775)	C = 0.1 (R^2: 0.910)	C = 1 (R^2: 0.928)	C = 10 (R^2: 0.932)	C = 100 (R^2: 0.933)	C = 1000 (R^2: 0.933)
x_1	0.0042	0.0087	0.0558	0.0881	0.1246	0.1597	0.1669
x_2	0.1052	0.4093	0.7472	0.9413	0.8739	0.7813	0.7634
x_1^2	0.0388	0.2178	0.4991	0.8433	1.0637	1.0768	1.0749
x_1 x_2	0.0008	0.0038	0.0225	0.1253	0.2516	0.3183	0.3302
x_2^2	-0.0075	-0.0306	-0.0340	0.0218	0.0832	0.0814	0.0795
x_1^3	0.0008	-0.0149	-0.0446	-0.0058	0.1108	0.1092	0.1026
x_1^2 x_2	0.0340	0.1169	0.1011	0.0987	0.2901	0.5005	0.5432
x_1 x_2^2	0.0046	0.0123	0.0246	-0.0397	-0.4551	-0.7825	-0.8414
x_2^3	0.0638	0.2171	0.1953	0.1787	0.5320	0.8681	0.9313
x_1^4	0.0353	0.1901	0.3287	0.1184	-0.0750	-0.0868	-0.0851
x_1^3 x_2	0.0017	0.0089	0.0219	-0.0102	-0.1210	-0.1987	-0.2132
x_1^2 x_2^2	0.0117	0.0552	0.0667	-0.0052	-0.0513	-0.0216	-0.0137
x_1 x_2^3	0.0007	-0.0058	-0.0721	-0.2230	-0.3445	-0.3897	-0.3973
x_2^4	-0.0099	-0.0400	-0.0530	-0.0849	-0.1382	-0.1479	-0.1485
x_1^5	-0.0007	-0.0241	-0.0975	-0.2318	-0.4434	-0.4902	-0.4923
x_1^4 x_2	0.0214	0.0698	0.0232	-0.0685	-0.2028	-0.3345	-0.3614
x_1^3 x_2^2	0.0031	0.0058	0.0281	0.2788	0.7752	1.0520	1.0999
x_1^2 x_2^3	0.0194	0.0571	-0.0305	-0.1887	-0.4378	-0.6808	-0.7286
x_1 x_2^4	0.0041	0.0111	0.0047	-0.0265	0.1869	0.4349	0.4815
x_2^5	0.0460	0.1430	0.0220	-0.1852	-0.4923	-0.7507	-0.7987

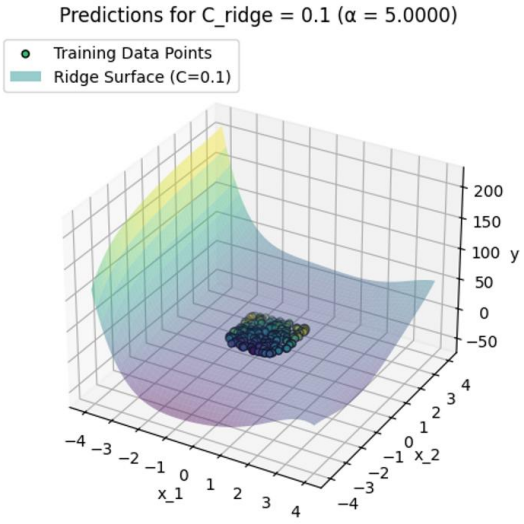
[Image 3: Table of coefficients that shows the features]

[Figure 5: Ridge predictions (surface) and training data (points) for C = [0.001], [0.01]]



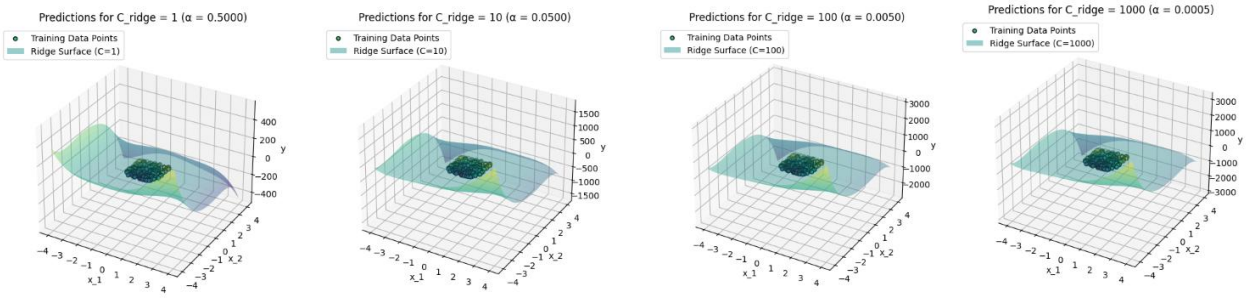
Caption: These figures perform underfitting

[Figure 6: Ridge predictions (surface) and training data (points) for C = [0.1]]



Caption: The most balanced model

[Figure 7: Ridge predictions (surface) and training data (points) for C = [1],[10],[100],[1000]]



Caption: These figures show the performance of overfitting models

Here the underfitting and overfitting groups between both Lasso and Ridge are different. The table below goes over this carefully:

Comparison field	Lasso	Ridge
Mathematical Penalty Type	L1 penalty, adds the absolute value of the coefficients	L2 penalty, adds the squared values of the coefficients
Treating coefficients	Drastically drops the irrelevant features and weights to exactly 0.0	Shrinks feature weights to 0 but keeps them all legible.
Best R^2 score	0.928	0.933
Underfitting performance	Like a rigid horizontal plane because it zeroes out everything.	Curves into a symmetrical rigid bowl because of non-zero values
Overfitting performance	Complicated geometric designs in the outer boundaries as features suddenly drop off to 0	Creates smooth wave like designs that explode violently at the edges

(ii) Choosing C with Cross-Validation

(ii)(a) 5-fold cross-validation for Lasso

The data was split into 5 folds- which are distinct equal sized subsets of the main dataset. For each C, the model was trained on 4 while the remaining fold was held out to evaluate predictive performance such a measuring error. This process was repeated 5 times so that every fold was treated in the training at least once. After this, the mean and standard deviation were taken across the folds, for each C. Here's a snippet:

```
C_values = np.logspace(-3, 5, 100)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
mse_scores_lasso = []
std_scores_lasso = []
. . .
. . .


for C in C_values:
. . .
    mse_fold.append(mean_squared_error(y_test, y_pred))

  mse_scores_lasso.append(np.mean(mse_fold))
  std_scores_lasso.append(np.std(mse_fold))
```

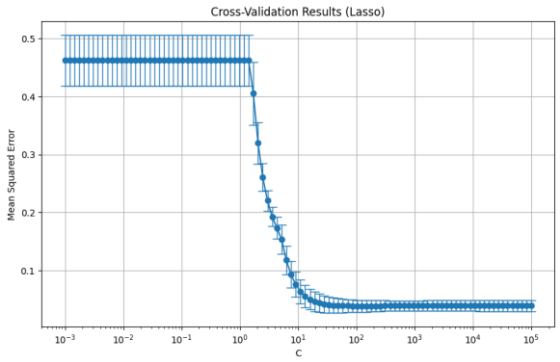
By tweaking around in the graph, my C range was found to be 10^{-3} to 10^{-5} is where all parameters are zero. It successfully captures both limits of regularized linear models. The highest boundary is where the error has levelled off or the model is c overfitting. From the lower boundary, the α is massive (α is less than 1.0) so Lasso forcefully drives all polynomial feature coefficients to exactly zero. As a result, the cross-validation error flatlines at its maximum baseline height. As C approaches 10^5, the penalty shrinks close to zero and leaves the 5th degree polynomial unconstrained. The cross-validation error completely falls to the bottom. The `DummyRegressor(strategy="mean"))` code was applied here:

```
baseline = DummyRegressor(strategy='mean')
baseline.fit(X_poly, y)
y_pred_baseline = baseline.predict(X_poly)
mse_baseline = mean_squared_error(y, y_pred_baseline)

print(f"Baseline MSE: {mse_baseline}")
```

This returned `Baseline MSE: 0.45420267737994374`

[Figure 8: Lasso 5-fold CV error (mean with std bars) vs C]



Caption: Mean Squared Error (MSE) profile with ± 1 standard deviation error bars across a logarithmic range of C configurations for Lasso regression.

(ii)(b) Recommended C for Lasso

In order to choose the best value of C, the lowest point in the plot was used. Where the mean error is the lowest, the bigger the standard deviation bars. For this I used the code:

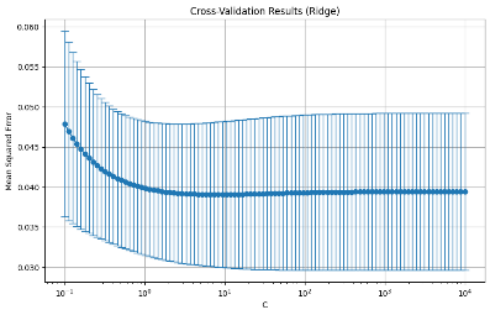
```
min_mse= min(mse_scores_lasso)
index_min = mse_scores_lasso.index(min_mse)
min_c_lasso = C_values[index_min]
```

This returned `123.28467394420684.` For architectural simplicity this centres perfectly within the optimal C= 100 order-of-magnitude window, where the model successfully minimizes Mean Squared Error before regularisation penalty

(ii)(c) Repeat for Ridge

The same was done with ridge but the model type was changed to deal with L2 regularization. The range validated was considered between 10^{-1} to 10^{4}. Under this the model acts rigidly, underfitting with the high mean standard error of 0. When C = 10^{-2} the regularization penalty α is excessively large. In this zone, the model is so severely constrained that it has completely flatlined into a massive underfitting state, causing the Mean Squared Error (MSE) to spike uncontrollably up to 0.12

[Figure 9: Ridge 5-fold CV error (mean with std bars) vs C]



Caption: Mean Squared Error (MSE) profile with ± 1 standard deviation error bars across a logarithmic range of C configurations for Ridge regression.

Here the C chosen is 8.30 which is the lowest point in the MSE standard graph. As how it was for Lasso, To the left of 8.30 (smaller C, larger α), the error curve turns sharply upward. This is the underfitting zone, where the penalty is too aggressive and strips away the model's ability to capture the data's true curvature. To the right of 8.30 (larger C, smaller α), the mean error curve stops dropping and subtly begins to flatline or edge slightly upward, while the standard deviation error bars remain wide. This is the entrance to the overfitting area.

Fields	Lasso	Ridge
Chosen C	123.28467394420684	8.302175681319744
Baseline (mean of y)	Same	Same
Non-zero parameters	10/20	20/20
R score	High	Higher than Lasso
Penalty (1/2C)	0.003289666123287841	0.06022517701293912

Appendix: Code

```
# # -*- coding: utf-8 -*-
"""assignment2ml.ipynb

Automatically generated by Colab.

"""

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from pathlib import Path

# look for assignment2.csv next to this script; fall back to the current folder
try:
    data_dir = Path(__file__).resolve().parent   # works when run as a .py file
except NameError:
    data_dir = Path.cwd()                        # works in a notebook / Colab

df = pd.read_csv("assignment2.csv", header=None)

#preview data
print(df.head())

# Extract features and target columns smoothly
X1 = df.iloc[:, 0].to_numpy()
X2 = df.iloc[:, 1].to_numpy()
X = np.column_stack((X1, X2))
y = df.iloc[:, 2].to_numpy()

print("--- Successfully Loaded Pure CSV! ---")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")

from mpl_toolkits.mplot3d import Axes3D

# initiliaze plot figure
fig= plt.figure(figsize=(8,6))
```

```
# plot
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X1, X2, y, c=y, cmap='viridis')

# format labels for clarity
ax.set_xlabel('x_1', fontsize = 10, labelpad =5)
ax.set_ylabel('x_2', fontsize = 10, labelpad =5)
ax.zaxis.set_rotate_label(False)
ax.set_zlabel('Y (target)', rotation=90)
ax.set_box_aspect(None, zoom=0.85)
ax.set_title('Data Plot', fontsize = 15)

# show plot
plt.legend()
plt.show()

"""In addition to the two features in the data file add extra polynomial features
equal to all combinations of powers of the two features up to power 5 (you
can use the sklearn PolynomialFeatures function to do this). Now train Lasso
regression models with these polynomial features for a large range of values of
C e.g. 1, 10, 1000 (you might need to adjust these values for your data, start
by making C small enough that the trained model has all parameters zero, then
increase from there). Report the parameters of the trained models (don't just
give a list of numbers, say what feature each parameter value corresponds to),
discuss how they change as C is varied.
"""

# import sklearn packages for feature engineering
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import pandas as pd

#add extra polynomial features equal to all combinations of powers to two features up to
power 5
poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)

feature_names = poly.get_feature_names_out(['x_1', 'x_2'])
table_data = {'Feature': feature_names}

c_options = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

#train lasso regression models
lasso_models = {}
for C in c_options:
    alpha = 1 / (2 * C)
    lasso = Lasso(alpha=alpha, max_iter=10000)
    lasso.fit(X_poly, y)
    lasso_models[C] = lasso

    #formatting with pd
    formatted_coefs = [round(coef, 4) if abs(coef) > 1e-4 else 0.0 for coef in lasso.coef_]
    column_name = f"C = {C} (R²: {lasso.score(X_poly, y):.3f})"
    table_data[column_name] = formatted_coefs

    print(f"Trained C = {C} (alpha = {alpha:.4f}) -> Intercept: {lasso.intercept_:.4f}")

table_of_coef = pd.DataFrame(table_data)
table_of_coef.set_index('Feature', inplace=True)

#raw view
print(table_of_coef)
```

```
# Clean up the view by dropping rows where all coefficients are 0
table_of_coef_filtered = table_of_coef[(table_of_coef != 0).any(axis=1)]
print("\t\t\t\t\t\t\t\t\t--- FINAL COMPARISON TABLE ---")
print(table_of_coef_filtered)

"""For each of the models from (b) generate predictions for the target variable.
Generate these predictions on a grid of feature values. You can use a couple of
nested for loops for this.

This grid should extend beyond the range of values in the dataset e.g. if the
first feature in the dataset has values from 0 to 2 generate predictions for values
from -5 to 5 or thereabouts. Plot these predictions on a 3D data plot and also
show the training data. Adjust the grid range used for the predictions so that the
training data can still be clearly seen in the plot. Its up to you to decide how best
to plot this data but do try to make your plot easy to read (suggestion: it can
be helpful to plot the predictions as a surface using the matplotlib plot surface
command and the training data as points using the matplotlib scatter command,
be sure to add a legend to identify the different curves). With reference to this
plot discuss how the predictions change as C is varied.
"""

# generate predictions using loops

for C, model in lasso_models.items():
    # Create a grid of feature values
    x1_min, x1_max = X[:, 0].min() - 3, X[:, 0].max() + 3 #added buffers
    x2_min, x2_max = X[:, 1].min() - 3, X[:, 1].max() + 3

    # Generate 1D ranges for the grid
    x1_range = np.linspace(x1_min, x1_max, 50)
    x2_range = np.linspace(x2_min, x2_max, 50)

    # generate predictions
    grid_points = []
    for x1 in x1_range:
        for x2 in x2_range:
            grid_points.append([x1, x2])

    X_grid = np.array(grid_points)

    # Transform grid points
    X_grid_poly = poly.transform(X_grid)

    # apply predictions with y
    y_pred_grid = model.predict(X_grid_poly)

    # Reshape coordinates back into grid matrices for plotting
    x1_grid = X_grid[:, 0].reshape(len(x1_range), len(x2_range))
    x2_grid = X_grid[:, 1].reshape(len(x1_range), len(x2_range))
    y_pred_mesh = y_pred_grid.reshape(len(x1_range), len(x2_range))

    # pkot
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')

    surface = ax.plot_surface(x1_grid, x2_grid, y_pred_mesh, cmap='viridis', alpha=0.4)

    scatter = ax.scatter(X[:, 0], X[:, 1], y, color='black', s=20, edgecolor='k',
label='Training Data')

    # Custom formatting
    ax.set_xlabel('x_1')
    ax.set_ylabel('x_2')
    ax.set_zlabel('y')
```

```
    ax.set_title(f'Predictions for C = {C} (α = {1/(2*C):.4f})')
    ax.set_box_aspect(None, zoom=0.8) #same as above

    # fix legend
    surface_patch = plt.Rectangle((0, 0), 1, 1, fc="purple", alpha=0.4)
    ax.legend([scatter, surface_patch], ['Training Data Points', f'Lasso Surface (C={C})'],
loc='upper left')

    plt.show()

"""What is under- and over-fitting? Using your parameter data from (b) and vi sualisation
from (c) explain how penalty weight parameter C can be used to manage to trade-off between
under- and over-fitting the data.

Repeat (b)-(c) for a Ridge Regression model. This uses an L2 penalty instead of
an L1 penalty in the cost function. Compare the impact on the model parameters of changing C
with Lasso Regression and with Ridge Regression
"""

# train with ridge regression
from sklearn.linear_model import Ridge
import matplotlib.pyplot as plt
import numpy as np

#add extra polynomial features equal to all combinations of powers to two features up to
power 5
poly_ridge = PolynomialFeatures(degree=5, include_bias=False)
X_poly_r = poly_ridge.fit_transform(X)

feature_names_ridge = poly_ridge.get_feature_names_out(['x_1', 'x_2'])
table_data_ridge = {'Feature': feature_names_ridge}

c_options = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

#train ridge regression models
ridge_models = {}
for C in c_options:
    alpha_ridge = 1/(2*C)
    ridge = Ridge(alpha=alpha_ridge, max_iter=10000)
    ridge.fit(X_poly_r, y)
    ridge_models[C] = ridge

    #formatting with pd
    formatted_coefs_ridge = [round(coef, 4) if abs(coef) > 1e-4 else 0.0 for coef in
ridge.coef_]
    column_name_ridge = f"C = {C} (R²: {ridge.score(X_poly_r, y):.3f})"
    table_data_ridge[column_name_ridge] = formatted_coefs_ridge

    print(f"Trained C = {C} (alpha = {alpha_ridge:.4f}) -> Intercept:
{ridge.intercept_:.4f}")

table_of_coef_ridge = pd.DataFrame(table_data_ridge)
table_of_coef_ridge.set_index('Feature', inplace=True)

#raw view
print(table_of_coef_ridge)

# Clean up the view by dropping rows where all coefficients are 0
table_of_coef_filtered_ridge = table_of_coef_ridge[(table_of_coef_ridge != 0).any(axis=1)]
print("\t\t\t\t\t\t\t\t\t--- FINAL COMPARISON TABLE ---")
print(table_of_coef_filtered_ridge)

# Loop through each trained Ridge model
for C_ridge, model_ridge in ridge_models.items():
```

```
    x1_min_ridge, x1_max_ridge = X[:, 0].min() - 3, X[:, 0].max() + 3 #added buffers
    x2_min_ridge, x2_max_ridge = X[:, 1].min() - 3, X[:, 1].max() + 3

    # Generate 1D ranges for the grid
    x1_vals_ridge = np.linspace(x1_min_ridge, x1_max_ridge, 50)
    x2_vals_ridge = np.linspace(x2_min_ridge, x2_max_ridge, 50)

    # generate predictions
    grid_points_ridge = []
    for x1 in x1_vals_ridge:
        for x2 in x2_vals_ridge:
            grid_points_ridge.append([x1, x2])

    X_grid_ridge = np.array(grid_points_ridge)

    # Transform grid points into Ridge's 5th-degree polynomial features
    X_grid_poly_ridge = poly_ridge.transform(X_grid_ridge)

    # apply predictions with y
    y_pred_grid_ridge = model_ridge.predict(X_grid_poly_ridge)

    # Reshape the predictions and coordinates back into matrices
    x1_grid_ridge = X_grid_ridge[:, 0].reshape(len(x1_vals_ridge), len(x2_vals_ridge))
    x2_grid_ridge = X_grid_ridge[:, 1].reshape(len(x1_vals_ridge), len(x2_vals_ridge))
    y_pred_mesh_ridge = y_pred_grid_ridge.reshape(len(x1_vals_ridge), len(x2_vals_ridge))

    # plot
    fig_ridge = plt.figure(figsize=(10, 6))
    ax_ridge = fig_ridge.add_subplot(111, projection='3d')

    scatter_ridge = ax_ridge.scatter(X[:, 0], X[:, 1], y, c=y, cmap='viridis', s=20,
edgecolor='k')

    surf_ridge = ax_ridge.plot_surface(x1_grid_ridge, x2_grid_ridge, y_pred_mesh_ridge,
cmap='viridis', alpha=0.4, linewidth=0)

    # custom formatting
    ax_ridge.set_xlabel('x_1')
    ax_ridge.set_ylabel('x_2')
    ax_ridge.set_zlabel('y')

    alpha_val_ridge = 1 / (2 * C_ridge)
    ax_ridge.set_title(f'Predictions for C_ridge = {C_ridge} (α = {alpha_val_ridge:.4f})')
    ax_ridge.set_box_aspect(None, zoom=0.85)

    # legend
    proxy_surface_ridge = plt.Rectangle((0, 0), 1, 1, fc="teal", alpha=0.4)
    ax_ridge.legend([scatter_ridge, proxy_surface_ridge], ['Training Data Points', f'Ridge
Surface (C={C_ridge})'], loc='upper left')

    plt.show()

"""(ii) Using the Lasso model with polynomial features from (i) you'll now look at using
cross-validation to select C.
(a) Use 5-fold cross-validation to plot the mean and standard deviation of the pre diction
error vs C. Use the matplotlib errorbar function for this. You will need
to choose the range of values of C to plot, justify your choice.
"""

# cross validation to select c
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error
import numpy as np
import matplotlib.pyplot as plt
```

```
# 5-fold cross validation to plot mean and standard deviation of prediction vs C
C_values = np.logspace(-3, 5, 100)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
mse_scores_lasso = []
std_scores_lasso = []

# loop and compare
for C in C_values:
    lasso = Lasso(alpha=1/(2*C), max_iter=10000)
    mse_fold = []
    for train_index, test_index in kf.split(X_poly):
        X_train, X_test = X_poly[train_index], X_poly[test_index]
        y_train, y_test = y[train_index], y[test_index]
        lasso.fit(X_train, y_train)
        y_pred = lasso.predict(X_test)
        mse_fold.append(mean_squared_error(y_test, y_pred))

    mse_scores_lasso.append(np.mean(mse_fold))
    std_scores_lasso.append(np.std(mse_fold))

# plot
plt.figure(figsize=(10, 6))
plt.errorbar(C_values, mse_scores_lasso, yerr=std_scores_lasso, fmt='-o', capsize=5,
elinewidth=1, markeredgewidth=1)
plt.xscale('log')
plt.xlabel('C')
plt.ylabel('Mean Squared Error')
plt.title('Cross-Validation Results (Lasso)')
plt.grid(True)
plt.show()

"""(b) Based on the cross-validation data what value of C would you recommend be
used here? Importantly, explain the reasons for your choice.
"""

min_mse= min(mse_scores_lasso)
index_min = mse_scores_lasso.index(min_mse)
min_c_lasso = C_values[index_min]
print(min_c_lasso)
penalty = 1/(2*min_c_lasso)
print(penalty)

# set up a baseline, the mean of y and the dummy regressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_squared_error

baseline = DummyRegressor(strategy='mean')
baseline.fit(X_poly, y)
y_pred_baseline = baseline.predict(X_poly)
mse_baseline = mean_squared_error(y, y_pred_baseline)

print(f"Baseline MSE: {mse_baseline}")

"""(c) Repeat (b)-(c) for a Ridge Regression model."""

# doing the same for ridge

# 5-fold cross validation to plot mean and standard deviation of prediction vs C
C_values = np.logspace(-1, 4, 100)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
mse_scores_ridge = []
std_scores_ridge = []

# loop and compare
for C in C_values:
```

```
    ridge= Ridge(alpha=1/(2*C), max_iter=10000)
    mse_fold = []
    for train_index, test_index in kf.split(X_poly_r):
        X_train, X_test = X_poly_r[train_index], X_poly_r[test_index]
        y_train, y_test = y[train_index], y[test_index]
        ridge.fit(X_train, y_train)
        y_pred = ridge.predict(X_test)
        mse_fold.append(mean_squared_error(y_test, y_pred))

    mse_scores_ridge.append(np.mean(mse_fold))
    std_scores_ridge.append(np.std(mse_fold))

# plot
plt.figure(figsize=(10, 6))
plt.errorbar(C_values, mse_scores_ridge, yerr=std_scores_ridge, fmt='-o', capsize=5,
elinewidth=1, markeredgewidth=1)
plt.xscale('log')
plt.xlabel('C')
plt.ylabel('Mean Squared Error')
plt.title('Cross-Validation Results (Ridge)')
plt.grid(True)
plt.show()

min_mse_ridge= min(mse_scores_ridge)
index_min_ridge = mse_scores_ridge.index(min_mse_ridge)
min_c_ridge = C_values[index_min_ridge]
print(min_c_ridge)
penalty_r = 1/(2*min_c_ridge)
print(penalty_r)
```